ЛЕКЦИЯ 4. КРОССПЛАТФОРМЕННАЯ РАЗРАБОТКА

РЫНОК МОБИЛЬНЫХ ОС (2025)

- **Android**: 70-75% мирового рынка
- **iOS**: 25-30%
- Android: ~3.5 млн приложений (Google Play)
- iOS: ~1.8 млн приложений (App Store)
- Альтернативы: HarmonyOS, KaiOS, Аврора

Поддерживать две отдельные кодовые базы - дорого и неэффективно.

ЧТО ТАКОЕ КРОССПЛАТФОРМЕННАЯ РАЗРАБОТКА?

Кроссплатформенная разработка - подход, при котором одно приложение пишется один раз, а затем запускается на нескольких платформах с минимальными доработками.

НЕДОСТАТКИ

ПРЕИМУЩЕСТВА

- Одна кодовая база
- Единая команда
- Быстрее на рынок
- Единообразный дизайн

- Производительность (для тяжёлых приложений)
- "Не совсем нативный" вид
- Зависимость от третьей стороны
- Размер приложения

ПОДХОДЫ К КРОССПЛАТФОРМЕННОЙ РАЗРАБОТКЕ

WEBVIEW-РЕНДЕРИНГ

- Cordova / Ionic
- HTML/CSS/JS в WebView
- Низкая производительность

НАТИВНЫЙ РЕНДЕРИНГ

- React Native (桥)
- Flutter (собственный движок)
- Высокая производительность

КОМПИЛЯЦИЯ В НАТИВНЫЙ

ВЕБ-ПРИЛОЖЕНИЯ

PWA (PROGRESSIVE WEB APP)

PWA: ЧТО ЭТО?

- **PWA** - веб-сайт, который ведёт себя как нативное приложение
- Установка на главный экран без App Store
- Работа в offline (Service Worker)
- Push-уведомления
- Не требует установки, автоматически обновляется
- Индексируется поисковиками (SEO)

PWA: SERVICE WORKER

```
// sw.js
const CACHE_NAME = 'v1';
const urlsToCache = [
    '/',
    '/styles.css',
    '/app.js',
    '/offline.html'
];

// Установка: кэширование ресурсов
self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => cache.addAll(urlsToCache))
    );
});

// Перехват запросов
self.addEventListener('fetch', event => {
    event.respondWith(
```

```
        caches.match(event.request)
            .then(response => {
                if (response) return response;
                return fetch(event.request).catch(() => {
                    return caches.match('/offline.html');
                });
            })
    );
});

// Активация: удаление старого кэша
self.addEventListener('activate', event => {
    event.waitUntil(
        caches.keys().then(keys =>
            Promise.all(keys.filter(k => k !== CACHE_NAME).map(k =>
        )
    );
});
```

PWA: WEB APP MANIFEST

```
// manifest.json
{
    "name": "Моё приложение",
    "short_name": "App",
    "description": "Описание приложения",
    "start_url": "/",
    "display": "standalone",
    "background_color": "#ffffff",
    "theme_color": "#6200EE",
    "orientation": "portrait",
    "scope": "/",
    "lang": "ru",
    "categories": ["productivity", "utilities"],
    "icons": [
        { "src": "/icon-192.png", "sizes": "192x192", "type": "imag
        { "src": "/icon-512.png", "sizes": "512x512", "type": "imag
        { "src": "/icon-maskable.png", "sizes": "512x512", "type":
    ],
    "screenshots": [
        { "src": "/screenshot.png", "sizes": "1080x1920", "type": "
```

```
    ]
}
```

PWA: PUSH-УВЕДОМЛЕНИЯ

```
// Регистрация Service Worker и подписка
if ('serviceWorker' in navigator && 'PushManager' in window) {
    const registration = await navigator.serviceWorker.registe

    const subscription = await registration.pushManager.subscr
        userVisibleOnly: true,
        applicationServerKey: urlBase64ToUint8Array('VAPID_KEY
    });

    // Отправка подписки на сервер
    await fetch('/api/push/subscribe', {
        method: 'POST',
        body: JSON.stringify(subscription),
        headers: { 'Content-Type': 'application/json' }
    });
}

// Обработка уведомления в sw.js
self.addEventListener('push', event => {
```

```
    const data = event.data.json();
    self.registration.showNotification(data.title, {
        body: data.body,
        icon: '/icon-192.png',
        badge: '/badge-72.png'
    });
});
```

PWA: ПРЕИМУЩЕСТВА И ОГРАНИЧЕНИЯ

ПРЕИМУЩЕСТВА

- Не нужен App Store
- Всегда обновляется
- Маленький размер
- SEO-дружелюбный
- Кроссплатформенно

ОГРАНИЧЕНИЯ

- Ограниченный доступ к ОС
- iOS: ограничена поддержка Push
- Нет в App Store / Google Play
- Ограничения по производительности
- Нет доступа к некоторым API

WEBVIEW

WEBVIEW

- **WebView** - встроенный браузерный компонент внутри нативного приложения
- Используется в гибридных приложениях
- Варианты использования:
  - Cordova/Ionic (обёртка веб-приложений)
  - Отображение HTML-контента в нативном приложении
  - Платёжные формы, виджеты

WEBVIEW: ПРИМЕР

```
// Android
class WebViewActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val webView = WebView(this).apply {
            settings.javaScriptEnabled = true
            settings.domStorageEnabled = true
            webViewClient = WebViewClient()
            loadUrl("https://example.com")
        }
        setContentView(webView)
    }
}

// iOS (Swift)
let webView = WKWebView(frame: view.bounds)
let url = URL(string: "https://example.com")!
webView.load(URLRequest(url: url))
```

```
view.addSubview(webView)
```

CORDOVA / IONIC

CORDOVA

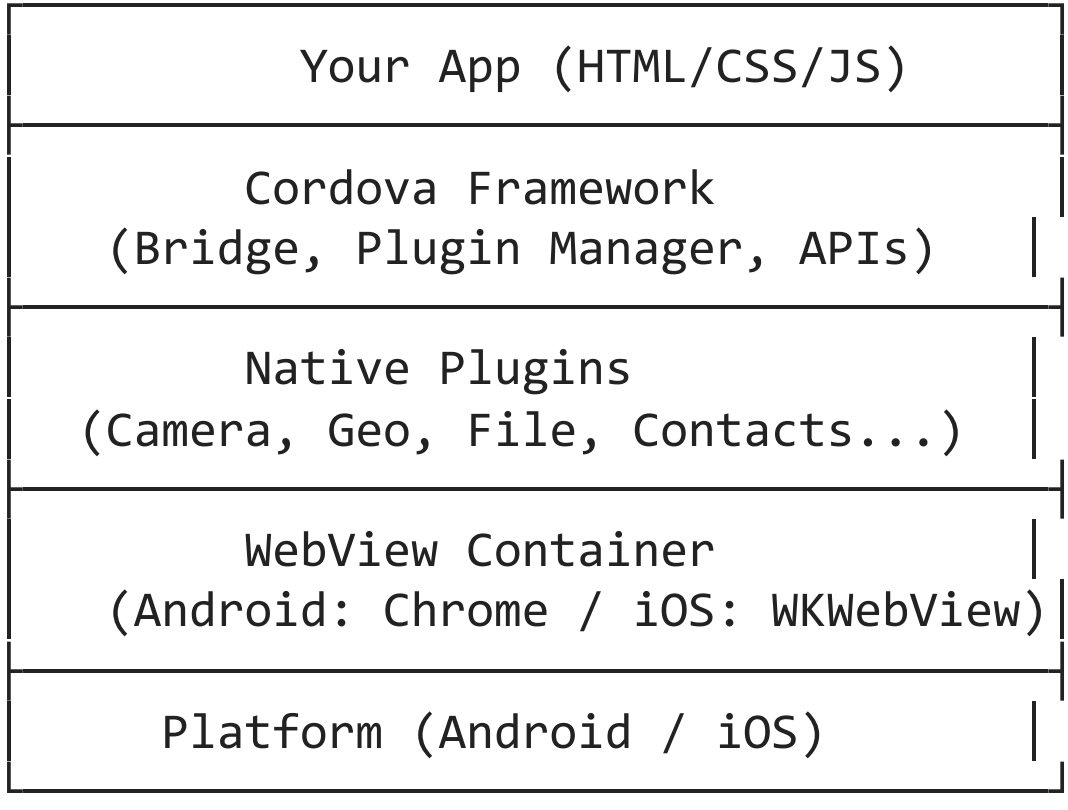
- Появился в 2009 (как PhoneGap)
- Технологии: HTML, CSS, JavaScript
- Рендеринг через WebView
- Плагины для доступа к нативным функциям
- Поддержка: Android, iOS, Windows

```
# Создание проекта
cordova create myApp com.example.myapp MyApp
cd myApp
cordova platform add android
cordova build android
cordova run android

# Установка плагинов
cordova plugin add cordova-plugin-camera
```

```
cordova plugin add cordova-plugin-geolocation
```

CORDOVA: АРХИТЕКТУРА



IONIC

- Фреймворк поверх Cordova/Capacitor
- UI-компоненты: Angular, React, Vue
- Material Design стили
- **Ionic Capacitor** - современная замена Cordova

```
# Создание Ionic-приложения
ionic start myApp tabs --type=react
cd myApp

# Запуск в браузере
ionic serve

# Сборка APK
ionic build
ionic capacitor sync android
```

```
ionic capacitor open android
```

IONIC CAPACITOR VS CORDOVA

Особенность	Cordova	Capacitor
Разработчик	Apache	Ionic
Подход	Обёртка	Нативные проекты доступны
Плагины	cordova-plugin-*	@capacitor-*
Native SDK	Не доступен	Полный доступ
Live Reload	Ограничен	Полная поддержка
Web-платформы	Нет	Да (PWA)

XAMARIN / .NET MAUI

XAMARIN

- Язык: **C#**
- Совместная бизнес-логика, отдельный UI для каждой платформы
- Доступ к нативным API через C#
- Xamarin.Forms - кроссплатформенный UI
- С 2024: **.NET MAUI** (successor)

```
// .NET MAUI
public class MainPage : ContentPage
{
    public MainPage()
    {
        Content = new StackLayout
        {
```

```
            Children = {
                new Label { Text = "Hello, .NET MAUI!" }
            }
        };
    }
}
```

.NET MAUI VS XAMARIN.FORMS

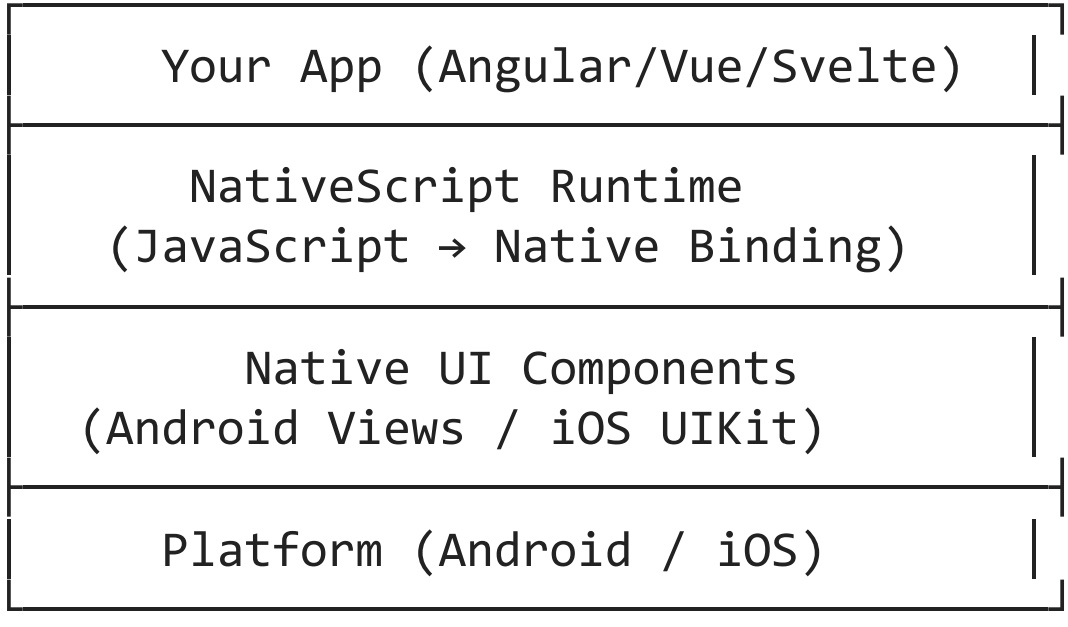
Особенность	Xamarin.Forms	.NET MAUI
.NET версия	.NET Standard	.NET 8+
Hot Reload	Ограниченный	Полный
Single Project	3 проекта	1 проект
Desktop	Нет	macOS, Windows
Продуктивность	Средняя	Высокая

NATIVESCRIPT

NATIVESCRIPT

- Язык: JavaScript/TypeScript
- Фреймворки: Angular, Vue, Svelte
- Прямой доступ к нативным API без обёрток
- Нативный рендеринг (не WebView)
- С 2024: NativeScript 8 (новая архитектура)

NATIVESCRIPT: АРХИТЕКТУРА




```
// Прямой доступ к API
import { Frame, Page } from '@nativescript/core';
import * as geolocation from '@nativescript/geolocation';

const location = await geolocation.getCurrentLocation();
```

PWA: WORKBOX

- **Workbox** — библиотека Google для управления Service Workers
- Готовые стратегии кэширования
- Генерация Service Worker через конфигурацию
- Интеграция с bundler (Webpack, Vite)

```
import { precacheAndRoute } from 'workbox-precaching';
import { registerRoute } from 'workbox-routing';
import { CacheFirst, StaleWhileRevalidate } from 'workbox-strategie

// Предварительное кэширование
precacheAndRoute(self.__WB_MANIFEST);

// Стратегия для изображений (Cache First)
registerRoute(
    ({ request }) => request.destination === 'image',
    new CacheFirst({
```

```
        cacheName: 'images',
        plugins: [
            new ExpirationPlugin({ maxEntries: 50, maxAgeSeconds: 3
        ]
    })
);

// Стратегия для API (Stale While Revalidate)
registerRoute(
    ({ url }) => url.pathname.startsWith('/api'),
    new StaleWhileRevalidate({ cacheName: 'api-cache' })
);
```

PWA: OFFLINE-FIRST ПАТТЕРНЫ

CACHE FIRST

Сначала кэш → если нет, сеть

Для: статика, шрифты, картинки

NETWORK FIRST

Сначала сеть → если нет, кэш

Для: API, динамический контент

STALE WHILE REVALIDATE

Кэш + фоновое обновление

Для: данные, которые быстро меняются

```
// Cache First
if (cachedResponse) return cachedResponse;
const networkResponse = await fetch(event.request);
cache.put(event.request, networkResponse.clone());
return networkResponse;

// Network First
try { return await fetch(event.request); }
```

```
catch { return caches.match(event.request); }

// Stale While Revalidate
const cached = await caches.match(event.request);
fetch(event.request).then(response => {
    cache.put(event.request, response.clone());
});
return cached || fetch(event.request);
```

PWA: WEB SHARE API

- Вызов нативного диалога «Поделиться» из PWA
- Поддержка: Chrome, Edge, Safari (частично)
- Работает на Android, ограниченно на iOS

```
// Проверка поддержки
if (navigator.share) {
    // Поделиться ссылкой
    await navigator.share({
        title: 'Моё приложение',
        text: 'Посмотрите этот классный сайт!',
        url: 'https://example.com'
    });
}

// Поделиться файлами
const file = new File(['Hello'], 'hello.txt', { type: 'text/pl
await navigator.share({
```

```
    title: 'Файл',
    files: [file]
});

// Web Share Target API (принятие данных из другого приложения
// В manifest.json:
// "share_target": {
//     "action": "/share",
//     "method": "GET",
//     "params": { "title": "title", "text": "text", "url": "u
// }
```

PWA: BADGING API

- Отображение значка (badge) на иконке приложения
- Поддержка: Chrome (Android), Edge
- iOS: ограниченно (только точка без числа)

```
// Установка badge
if ('setAppBadge' in navigator) {
    // Показать число
    await navigator.setAppBadge(12);

    // Показать точку (без числа)
    await navigator.setAppBadge();

    // Убрать badge
    await navigator.clearAppBadge();
}
```

```
// В Service Worker
self.registration.setAppBadge(5);

// Использование: PWA-почта, мессенджеры, таск-трекеры
```

PWA: BACKGROUND SYNC

- Отправка данных, когда пользователь снова онлайн
- Работает даже после закрытия браузера
- Идеально для форм, сообщений, синхронизации

```
// Регистрация синхронизации
const registration = await navigator.serviceWorker.ready;
await registration.sync.register('send-form');

// Обработка в Service Worker
self.addEventListener('sync', event => {
    if (event.tag === 'send-form') {
        event.waitUntil(sendPendingData());
    }
});
```

```
async function sendPendingData() {
    const db = await openDB('pending-requests');
    const requests = await db.getAll('forms');

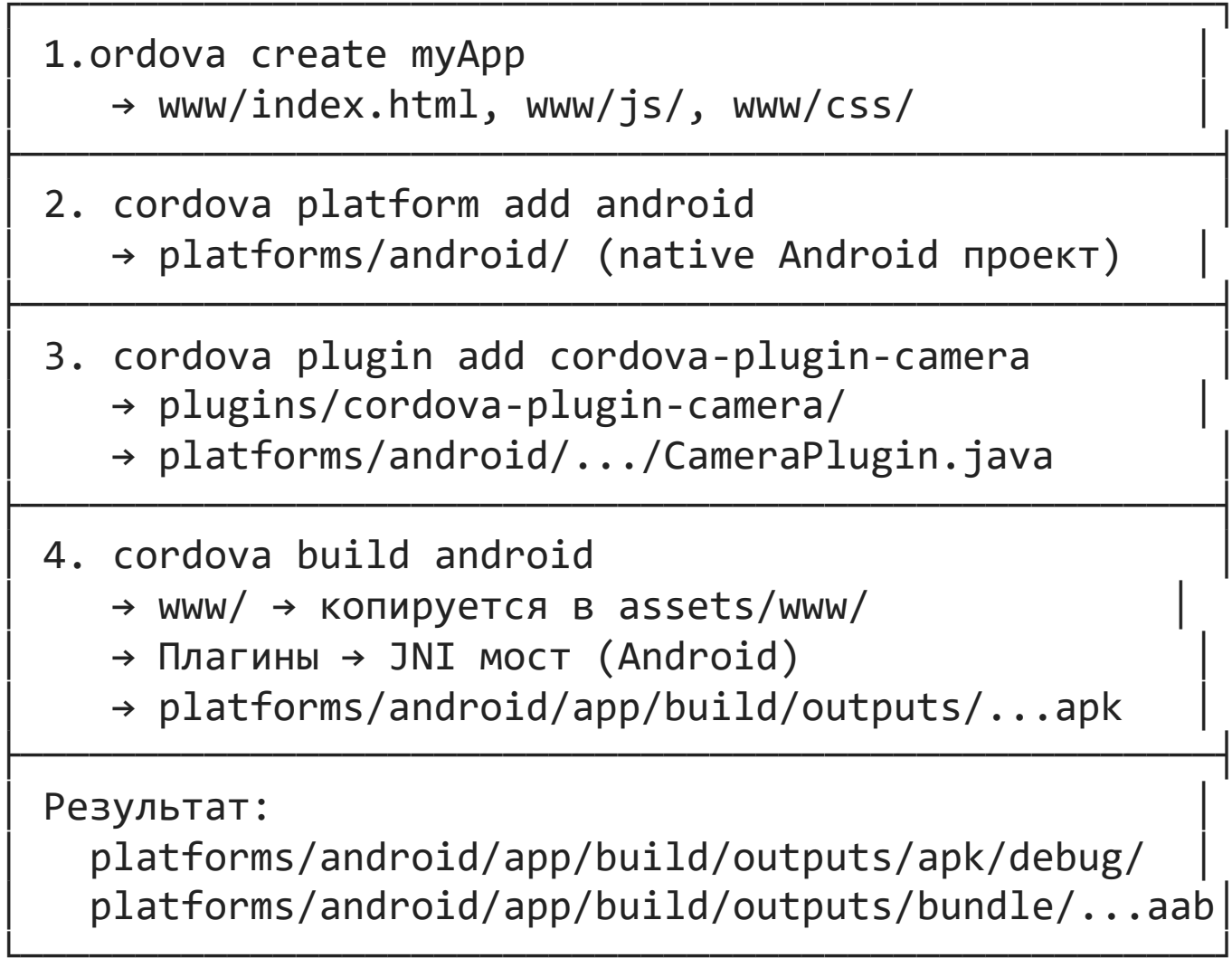
    for (const req of requests) {
        try {
            await fetch('/api/submit', {
                method: 'POST',
                body: JSON.stringify(req),
                headers: { 'Content-Type': 'application/json' }
            });
            await db.delete('forms', req.id);
        } catch (e) {
            // Повтор при следующей синхронизации
        }
    }
}
```

CORDOVA: ЭКОСИСТЕМА ПЛАГИНОВ

Плагин	Описание	Установка
Camera	Фото/видео с камеры	`cordova-plugin-camera`
Barcode Scanner	Сканирование штрих-кодов/QR	`cordova-plugin-barcodescanner`
Geolocation	GPS-координаты	`cordova-plugin-geolocation`
File	Доступ к файловой системе	`cordova-plugin-file`
Device	Информация об устройстве	`cordova-plugin-device`
Network	Статус сети	`cordova-plugin-network-information`
Vibration	Вибрация	`cordova-plugin-vibration`

Local Notifications	Локальные уведомления	`cordova-plugin-local-notification`
InAppBrowser	Открытие ссылок в браузере	`cordova-plugin-inappbrowser`

CORDOVA: ПРОЦЕСС СБОРКИ



WWW-директория копируется в нативный проект → WebView загружает HTML → Плагины связывают JS с нативным кодом.

IONIC: ОБЗОР КОМПОНЕНТОВ

ION-BUTTON

```
<ion-button
fill="solid">Click</ion-
button>
```

ION-CONTENT

```
<ion-content
padding>...</ion-
content>
```

ION-TA

```
<ion-tabs><ion-
</ion-tab-bar><
```

ION-LIST

```
<ion-list><ion-item>...
</ion-item></ion-list>
```

ION-MODAL

```
<ion-modal
trigger="open">...
</ion-modal>
```

ION-ALE

```
ionAlertCtrl.cr
```

Все компоненты стилизуются под платформу (iOS/Android автоматически).

IONIC: CAPACITOR НАТИВНЫЕ API

```
import { Filesystem, Directory } from '@capacitor/filesystem';
import { Camera, CameraResultType } from '@capacitor/camera';
import { LocalNotifications } from '@capacitor/local-notifications'

// Файловая система
const result = await Filesystem.writeFile({
    path: 'data.json',
    data: JSON.stringify({ users: [...] }),
    directory: Directory.Documents
});

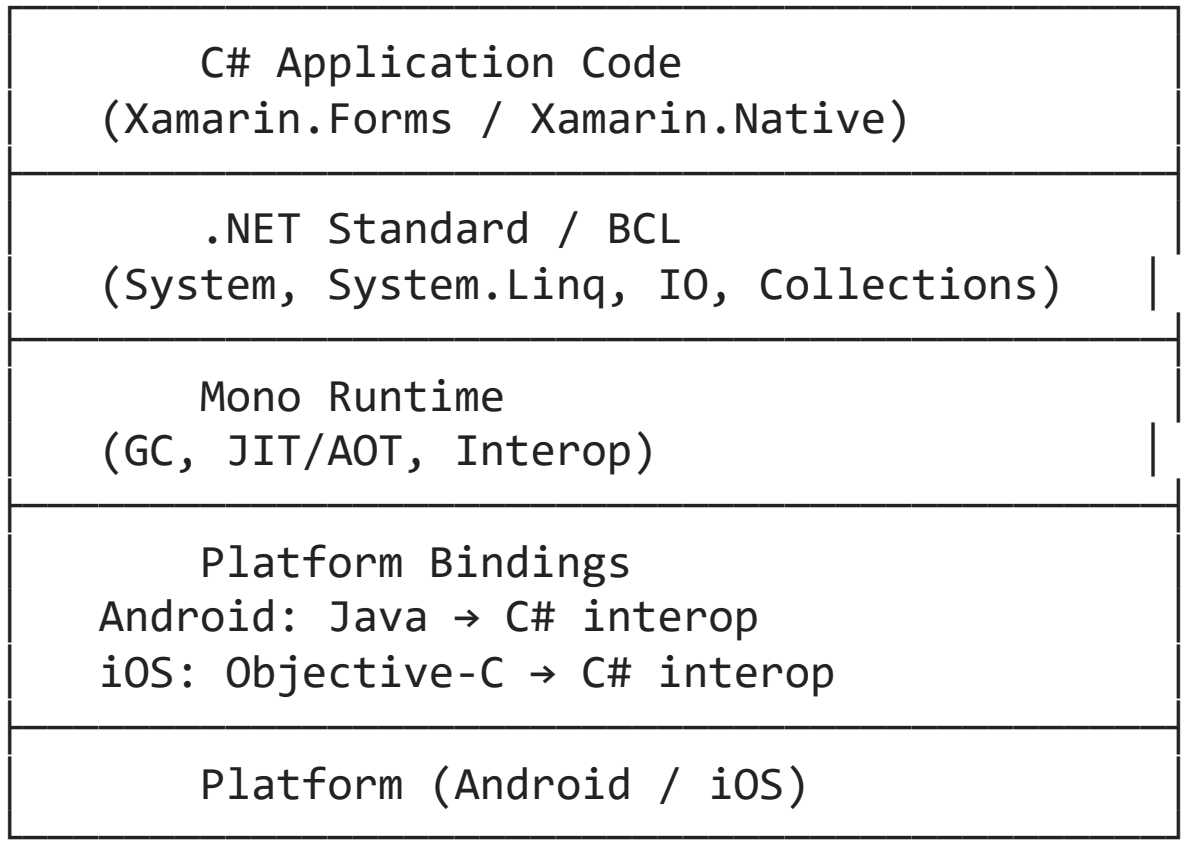
// Камера
const photo = await Camera.getPhoto({
    quality: 90,
    resultType: CameraResultType.Base64
});

// Локальные уведомления
await LocalNotifications.schedule({
    notifications: [{
```

```
        title: 'Напоминание',
        body: 'Не забудьте про встречу!',
        id: 1,
        schedule: { at: new Date(Date.now() + 3600000) }
    }]
});
```

Capacitor даёт полный доступ к нативному SDK: Filesystem, Camera, Notifications, Biometrics, Geolocation и др.

XAMARIN: АРХИТЕКТУРА



Mono Runtime:

- JIT-компиляция (Android)
- AOT-компиляция (iOS, обязательна)
- GC (Garbage Collector) автоматически

- P/Invoke для вызова нативных библиотек (C/C++)

Xamarin использует Mono Runtime для исполнения C# на мобильных платформах с доступом к нативным API через обёртки.

REACT NATIVE: EXPO

MANAGED WORKFLOW

- Expo CLI управляет проектом
- Нет доступа к нативному коду
- Быстрый старт: `npx create-expo-app`
- OTA-обновления через EAS
- 700+ плагинов Expo

BARE WORKFLOW

- Полный доступ к нативному коду
- Можно подключить любой npm-пакет
- Сложнее настройка
- Нужен Xcode / Android Studio
- Для продвинутых случаев

```
# Managed
npx create-expo-app MyApp
cd MyApp
npx expo start
```

```
# Bare
npx create-react-native-app MyApp --template bare
```

REACT NATIVE: НАВИГАЦИЯ

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native
import { createBottomTabNavigator } from '@react-navigation/bottom-

const Stack = createNativeStackNavigator();
const Tab = createBottomTabNavigator();

// Stack Navigation (экран → экран)
<NavigationContainer>
    <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
    </Stack.Navigator>
</NavigationContainer>

// Tab Navigation (нижние вкладки)
<Tab.Navigator>
    <Tab.Screen name="Feed" component={FeedScreen} />
    <Tab.Screen name="Profile" component={ProfileScreen} />
    <Tab.Screen name="Settings" component={SettingsScreen} />
```

```
</Tab.Navigator>

// Drawer Navigation (боковое меню)
// npm install @react-navigation/drawer
```

React Navigation — стандартная библиотека навигации: Stack, Tab, Drawer, Deep Linking.

REACT NATIVE: УПРАВЛЕНИЕ СОСТОЯНИЕМ

Библиотека	Подход	Размер	Когда использовать
Redux Toolkit	Centralized store	~11 KB	Сложные приложения, enterprise
Zustand	Flux-like, hooks	~1 KB	Простой API, быстрый старт
Jotai	Atomic (атомы)	~3 KB	Мелкозернистые обновления
Recoil	Atomic	~11 KB	Complex state графы
MobX	Observable	~16 KB	Reactive обновления

```
// Zustand — пример
import { create } from 'zustand';

const useStore = create((set) => ({
    count: 0,
    increment: () => set(state => ({ count: state.count + 1 })),
}));

// Использование
```

```
const { count, increment } = useStore();
```

REACT NATIVE: NEW ARCHITECTURE

JSI

JavaScript Interface

Замена Bridge

Синхронный вызов нативного кода

Удаление serialization/deserialization

FABRIC

Новый рендерер

Синхронное дерево компонентов

Concurrent mode

Приоритизация обновлений UI

TURBOMODULES

Ленивая загрузка

Загружаются при первом использовании

Type-safe (Codegen)

Уменьшение времени старта

FLUTTER: IMPELLER RENDERING ENGINE

- **Impeller** — новый движок рендеринга (замена Skia)
- Включён по умолчанию с Flutter 3.16+
- Предкомпиляция шейдеров → нет задержек (jank)
- Более предсказуемая производительность

Особенность	**Skia**	**Impeller**
Рендеринг	Rasterization	Tiled rendering
Шейдеры	Компилируются во время работы	Предкомпилированы
Jank	Возможен (первые кадры)	Минимален

FLUTTER: DART FFI

- **FFI (Foreign Function Interface)** — вызов C/C++ кода из Dart
- Для высокопроизводительных вычислений
- Интеграция с существующими C-библиотеками

```
import 'dart:ffi';
import 'package:ffi/ffi.dart';

// Определение C-функции
typedef AddNative = Int32 Function(Int32 a, Int32 b);
typedef AddDart = int Function(int a, int b);

// Загрузка библиотеки
final dylib = DynamicLibrary.open('libmath.so');
final add = dylib.lookupFunction<AddNative, AddDart>('add');

// Вызов
```

```
final result = add(2, 3); // 5

// Примеры использования:
// • OpenSSL для криптографии
// • SQLite для базы данных
// • FFmpeg для обработки видео
// • zlib для сжатия данных
```

FLUTTER: PLATFORM CHANNELS

- Связь между Dart и нативным кодом (Kotlin/Swift)
- Асинхронный обмен сообщениями
- Три типа каналов:

METHODCHANNEL

Вызов методов (RPC)

EVENTCHANNEL

Стрим данных (события)

BASICMESSAGECHANNEL

Обмен сообщениями

```
// Dart
final battery = await platform.invokeMethod('getBatteryLevel');

// Android (Kotlin)
MethodChannel("com.example/app").setMethodCallHandler { call ->
    when (call.method) {
        "getBatteryLevel" -> getBatteryLevel()
```

```
        else -> null
    }
}
```

СРАВНЕНИЕ: СКОРОСТЬ РАЗРАБОТКИ

Технология	Time to Market	Команда	Hot Reload
Нативная	8-12 недель	2+ команды	Да
Flutter	4-6 недель	1 команда	Sub-second
React Native	4-6 недель	1 команда	Fast Refresh
Ionic/Capacitor	3-5 недель	1 команда	Да
Cordova	3-5 недель	1 команда	Да
PWA	2-4 недели	1 команда	Авто
Xamarin/.NET MAUI	5-7 недель	1 команда	Hot Reload
NativeScript	4-6 недель	1 команда	Да

Кроссплатформенные решения ускоряют выход на рынок в 1.5-2 раза.

СРАВНЕНИЕ: РАЗМЕР ПРИЛОЖЕНИЙ

Технология	Hello World APK	Среднее приложение	Примечания
Нативная (Kotlin)	~1 MB	5-15 MB	Минимальный размер
Нативная (Swift)	~1 MB	8-20 MB	С Atlas-bitcode
Flutter	~7 MB	15-30 MB	Flutter engine включён
React Native	~7 MB	10-25 MB	JS bundle + Hermes
Ionic/Capacitor	~5 MB	10-20 MB	Веб-контент
Cordova	~5 MB	10-25 MB	Веб-контент + плагины
Xamarin	~15 MB	20-40 MB	.NET runtime включён
PWA	~100 KB	1-5 MB	Без установки

СРАВНЕНИЕ: СООБЩЕСТВО

Технология	GitHub Stars	SO Questions	Job Market
Flutter	165K+	120K+	Растущий спрос
React Native	120K+	180K+	Высокий спрос
Xamarin/MAUI	20K+	80K+	Средний (enterprise)
Ionic	50K+	60K+	Средний
Cordova	3.5K+	100K+	Снижается
NativeScript	23K+	15K+	Небольшой
PWA	—	40K+	Универсальный

Flutter и React Native — самые активные сообщества в кроссплатформенной разработке.

HARMONYOS

- **HarmonyOS** — операционная система Huawei
- Разработка: **ArkTS** (TypeScript-подобный) + **ArkUI** (декларативный UI)
- Цель: экосистема без Google Play Services
- Поддержка: смартфоны, планшеты, wearable, авто, IoT
- AppGallery — магазин приложений

```
// ArkTS (HarmonyOS)
@Entry
@Component
struct Index {
    @State message: string = 'Hello, HarmonyOS!';

    build() {
```

```
        Column() {
            Text(this.message)
                .fontSize(30)
                .fontWeight(FontWeight.Bold)
            Button('Click me')
                .onClick(() => {
                    this.message = 'Button clicked!';
                })
        }
        .width('100%')
        .height('100%')
        .justifyContent(FlexAlign.Center)
    }
}
```

KOTLIN COMPOSE MULTIPLATFORM

- **Compose Multiplatform** — декларативный UI на Kotlin (JetBrains)
- Расширение Jetpack Compose для кроссплатформенности
- Поддержка: Android, iOS, Desktop, Web (Wasm)
- Совместим с KMP (Kotlin Multiplatform) для общей логики

```
// commonMain – общий UI
@Composable
fun App() {
    var count by remember { mutableStateOf(0) }

    Column(
        modifier = Modifier.fillMaxSize(),
```

```
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text("Count: $count", style = MaterialTheme.typography.headl
        Button(onClick = { count++ }) {
            Text("Increment")
        }
    }
}

// Android: actual fun MainActivity() { setContent { App() } }
// iOS: actual fun MainViewController() { ComposeUIViewController {
// Desktop: fun main() = application { Window(onCloseRequest = ::ex
```

REACT NATIVE

REACT NATIVE

- Создан Meta (Facebook) в 2015
- Язык: JavaScript/TypeScript
- UI-компоненты через React
- Hot Reload
- Нативные модули для доступа к API
- Новый архитектурный подход: **JSI + Fabric + TurboModules** (2024)

JSX И REACT NATIVE

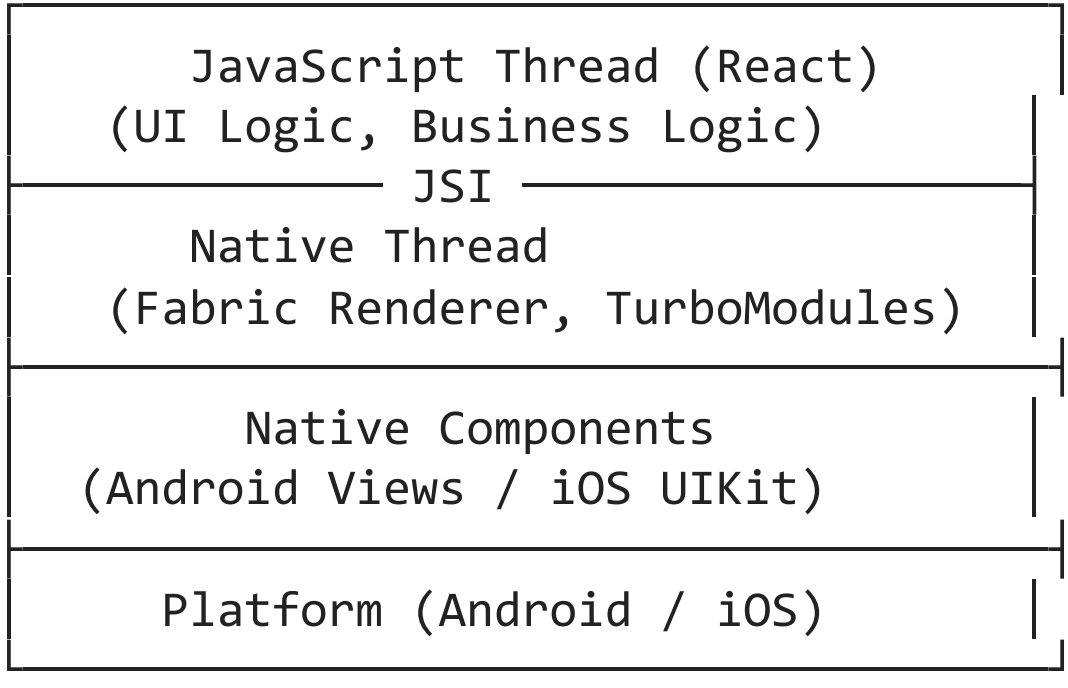
```
// JSX - расширение JavaScript для описания UI
const App = () => {
    const [count, setCount] = useState(0);

    return (
        <View style={styles.container}>
            <Text style={styles.title}>
                Счётчик: {count}
            </Text>
            <Button
                title="Нажми"
                onPress={() => setCount(count + 1)}
            />
        </View>
    );
};

const styles = StyleSheet.create({
    container: { flex: 1, justifyContent: 'center', alignItems: 'ce
    title: { fontSize: 24, marginBottom: 20 }
```

```
});
```

REACT NATIVE: АРХИТЕКТУРА




```
// Новая архитектура (2024):
// JSI: JavaScript Interface (замена Bridge)
// Fabric: новый рендеринг
// TurboModules: ленивая загрузка нативных модулей
```

FLUTTER: КРАТКИЙ ОБЗОР

FLUTTER

- Создан Google в 2018
- Язык: **Dart**
- Собственный рендеринг (Impeller) - не использует WebView
- Material Design + Cupertino виджеты
- Кроссплатформенно: Android, iOS, Web, Windows, macOS, Linux
- Hot Reload для быстрой разработки
- Подробно: следующие 3 лекции

FLUTTER: АРХИТЕКТУРА

FRAMEWORK	ENGINE	EMBEDDER
Material Design, Cupertino, Widgets, Animation	Impeller/Skia, Dart VM, Platform Channels	Связь с ОС (Android, iOS, Desktop)

СРАВНЕНИЕ ТЕХНОЛОГИЙ (2025)

Критерий	Нативная	React Native	Flutter	PWA	Cordova
Производительность	Высокая	Хорошая	Высокая	Средняя	Низкая
Размер приложения	Маленький	Средний	Средний	Минимальный	Средний
Доступ к API	Полный	Через модули	Через каналы	Ограниченный	Через плагины
Hot Reload	Есть	Есть	Есть	Авто	Есть
Скорость разработки	2 кодовые базы	1 кодовая база	1 кодовая база	1 кодовая база	1 кодовая база
Нативный вид	Да	Почти	Кастомный	Нет	Нет
Обучаемость	Сложно (2 языка)	JS/TS	Dart	HTML/CSS/JS	HTML/CSS/JS

КОГДА ЧТО ВЫБИРАТЬ?

Нативная разработка: тяжёлые игры, AR/VR, сложная графика, максимальная производительность

Flutter: стартапы, приложения с кастомным дизайном, multiplatform (включая desktop/web)

React Native: если команда знает React, быстрое прототипирование

PWA: простые приложения, каталоги, лендинги, минимальный бюджет

Cordova/Ionic: существующие веб-приложения, гибридные решения

ТРЕНДЫ КРОССПЛАТФОРМЕННОЙ РАЗРАБОТКИ (2025)

- **Flutter** - быстрый рост, поддержка desktop/web
- **React Native** - новая архитектура (Fabric, JSI)
- **Kotlin Multiplatform (KMP)** - общая логика на Kotlin
- **Compose Multiplatform** - UI на Kotlin (JetBrains)
- **Kotlin/Wasm** - компиляция в WebAssembly
- **AI-интеграция** - генерация UI, Copilot для мобильной разработки

KOTLIN MULTIPLATFORM (KMP)

- Общая бизнес-логика на Kotlin для Android/iOS/Desktop/Web
- UI остаётся нативным (Compose Multiplatform для Android, SwiftUI для iOS)
- Не замена Flutter/RN, а дополнение к нативной разработке

```
// Общий код (shared/src/commonMain)
class UserRepository {
    suspend fun getUsers(): List {
        return httpClient.get("https://api.example.com/use
    }
}
```

```
// Android (использование)
val users = repository.getUsers()

// iOS (использование через Ktor)
let users = try await repository.getUsers()
```